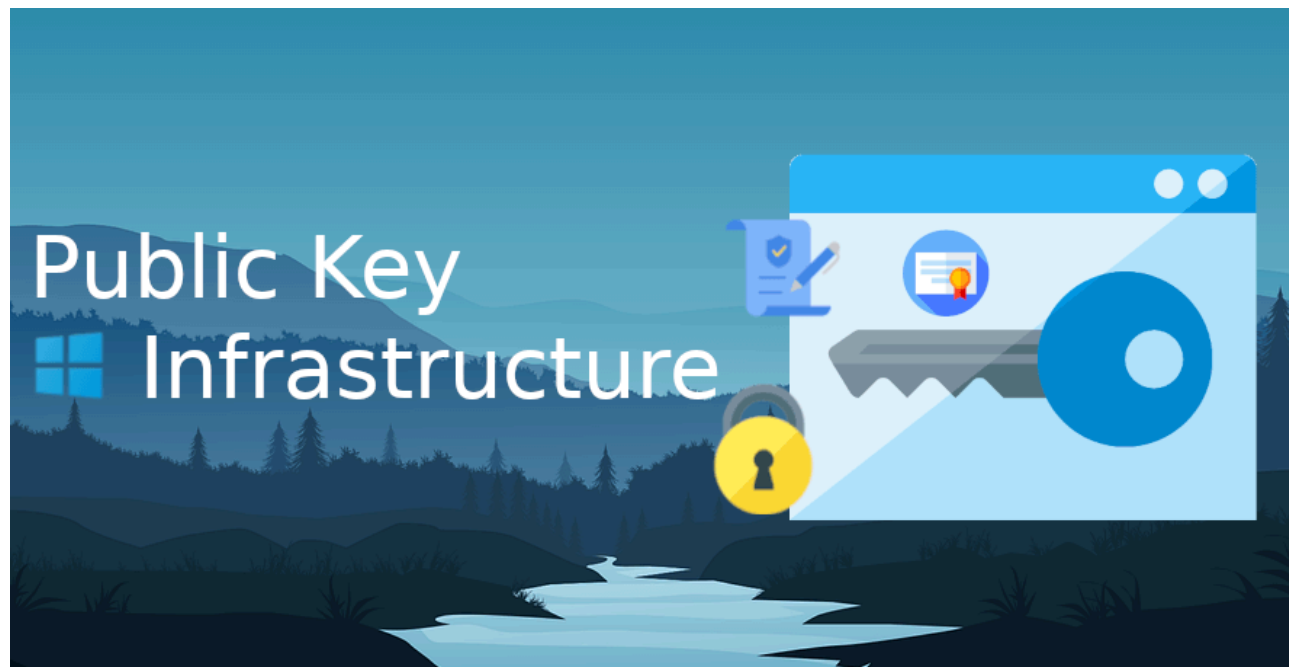


Build a PKI with PowerShell – Part 2 – IIS WebServer – Michael Waterman

 michaelwaterman.nl/2026/01/03/how-to-build-a-pki-with-powershell-part-2-iis-webserver

Michael Waterman

January 3, 2026



Other parts in this series

[How to: Build a PKI with PowerShell – Part 1 – Preparation](#)

[How to: Build a PKI with PowerShell – Part 3 – Offline Root CA](#)

[How to: Build a PKI with PowerShell – Part 4 – Enterprise CA](#)

In the [previous part](#), I've covered the design choices and preparation work needed before touching any infrastructure. In this part, I'll finally start building something: the *PKI Web Server*.

I know, I know, not the most exciting exercise, but stay tuned, perhaps I'll have some former Microsoft Security engineer tips here! However boring, this server plays a crucial role in the overall trust model. It hosts:

- The **Certificate Revocation List (CRL)**
- The **Certificate Distribution Point (CDP)**
- The **Certification Practice Statement (CPS)**

In short: it becomes the “*public-facing*” component of your PKI.

The role of the Web Server

Every certificate issued by your PKI contains references to where clients can:

- Check whether a certificate has been revoked
- Retrieve intermediate or root certificates
- Validate trust paths

These locations must be:

- Highly available
- Predictable
- Accessible without authentication

That's why this role is intentionally separated from the CA itself. The CA remains protected, while the web server acts as a controlled distribution point. Although this setup focuses on setting up a single server, it would be best to make this high-available. A simple setup with a load balancer where the dns entry "trust.<your domain>" points to could do the trick.

Step 1 – Preparing the server

Required permissions

- Local Administrator on the PKI Web Server
- No domain-level permissions required

Why: Installing IIS, configuring folders, shares, NTFS permissions, and IIS settings only requires local administrative rights on the web server. The server must be domain joined, but no Active Directory modifications are performed.

Before running any Commands, the server should also meet the prerequisites defined earlier:

- Windows Server installed, I use Windows Server 2025 Core as this is more than enough for hosting certificates.
- Network connectivity available
- DNS resolution in place (e.g. `trust.domainname.suffix`)

This server does *not* need to be domain-joined, but it can be, the design supports both. I do highly recommend that the machine is domain joined though, in that case the certificates revocation lists (CRL) will be automatically published to the CRL location. Having a stand-alone server means you will have to do this manually or via another automated way. Just so you know...

Step 2 – Installing IIS

The first technical step is installing Internet Information Services (IIS). This provides the web service used to host CRLs, CDPs, and the CPS.

Run the following command in an elevated PowerShell session:

```
Install-WindowsFeature Web-Server -IncludeManagementTools
```

Note! use `-IncludeManagementTools` only on a GUI based installation, not on Windows Server core.

Note! After installing, make sure you import the “*WebAdministration module*” into your session.

```
Import-Module "WebAdministration"
```

Step 3 – Creating the PKI directory structure

Next, we create a clean and predictable folder structure for PKI-related content.

```
New-Item -Path "C:\inetpub\cdp\crl" -ItemType Directory -force  
New-Item -Path "C:\inetpub\cdp\cps" -ItemType Directory -force
```

Each folder has a specific purpose:

- **CDP** (Certificate Deployment Point)– Used by clients to retrieve intermediate certificates
- **CRL** (Certificate Revocation List) – Used to check revocation status
- **CPS** (Certificate Policy Statement) – Contains documentation describing certificate usage and policy. Even if you don’t publish a full CPS yet, having a predictable CPS endpoint prevents “404” situations when CPS URLs are included in templates/policies.

Once certificates are issued, these paths become part of the certificate itself, well obviously not the SMB path but how DNS points to it, so consistency is critical.

Step 4 – Create an example CPS

```
New-Item -Path "C:\inetpub\cdp\cps\cps.html" -ItemType File  
Set-Content -Path "C:\inetpub\cdp\cps\cps.html" -Value '<html>Placeholder for the  
Certification Practice Statement</html>'
```

This is just a placeholder for your policy statement file, or the redirection to the document that describes the policy for the usage of your PKI environment,

Step 5 – Creating the IIS website

Now we create a dedicated IIS website for PKI distribution. This ensures clean separation from any other web services running on the system.

```
New-Website `
-Name "Certificate Distribution Point" `
-PhysicalPath "C:\inetpub\cdp" `
-HostHeader "trust.domain.suffix"
```

This cleanly separates PKI distribution from “Default Web Site” and binds it to a stable hostname (e.g., trust.domain.suffix) which is exactly what clients will use when validating certificates.

Step 6 – Publishing a validation endpoint

Before moving on, it’s useful to have a simple way to verify that everything works as expected.

Create a small text file that acts as a beacon, we’ll test it at a later point.

```
"PKI Beacon" | Out-File "C:\inetpub\cdp\beacon" -Encoding ascii
```

Step 7 – Create IIS virtual directories for CRL and CPS

```
New-WebVirtualDirectory -Site "Certificate Distribution Point" -Name "CRL" -
PhysicalPath "C:\inetpub\cdp\crl"
New-WebVirtualDirectory -Site "Certificate Distribution Point" -Name "CPS" -
PhysicalPath "C:\inetpub\cdp\cps"
```

This publishes clean URLs like:

- http://trust.<domain>/crl/...
- http://trust.<domain>/cps/cps.html

These are the URLs you’ll later embed into certificates, so make sure you get this right!

Step 8 – Allow “double escaping” in IIS request filtering

```
Set-WebConfigurationProperty -Filter "system.webServer/security/requestFiltering" `
-Name "allowDoubleEscaping" -Value $true `
-PSPath "IIS:\sites\Certificate Distribution Point"
```

PKI URLs (*especially CRL/AIA*) can include characters that IIS normally treats as suspicious (the plus sign (+) for example). This prevents IIS from rejecting valid PKI fetch requests due to strict URL parsing.

Step 9 – Configure client-side caching for static content

```
Set-WebConfigurationProperty -Filter "system.webServer/staticContent/clientCache" `
    -Name "cacheControlMode" -Value "UseMaxAge" -PSPath "IIS:\sites\Certificate
Distribution Point"
```

```
Set-WebConfigurationProperty -Filter "system.webServer/staticContent/clientCache" `
    -Name "cacheControlMaxAge" -Value "7.00:00:00" -PSPath "IIS:\sites\Certificate
Distribution Point"
```

```
Set-WebConfigurationProperty -Filter "system.webServer/staticContent/clientCache" `
    -Name "setEtag" -Value $true -PSPath "IIS:\sites\Certificate Distribution Point"
```

CRLs and related files are static content. Proper caching reduces unnecessary traffic while keeping refresh behavior predictable which is especially important for updating CRL files in a timely manner. The example above invalidates the cache after 7 days, this setting is variable, depending on your needs.

Step 10 – Add a MIME mapping to make the beacon reachable

```
Add-WebConfigurationProperty -Filter "system.webServer/staticContent" -Name "." `
    -Value @{ fileExtension='.'; mimeType='text/xml' } `
    -PSPath "IIS:\sites\Certificate Distribution Point"
```

Your beacon file has **no extension**. IIS may refuse to serve extensionless files unless configured. This ensures `.../beacon` is actually downloadable.

After this step, you can now test connectivity from another system by using a browser or PowerShell using `Invoke-WebRequest`.

```
Invoke-WebRequest -Uri http://trust.<domain.suffix>/beacon -UseBasicParsing |
select -expandproperty content
```

If this returns `PKI Beacon`, your web server is reachable and ready to be used by the PKI infrastructure.

Step 11 – Create the CRL distribution point SMB share

```
New-SmbShare -Name "crl" `
    -Path "C:\inetpub\cdp\crl" `
    -ReadAccess "Authenticated Users" `
    -FullAccess "Cert Publishers", ".\Administrators" `
    -CachingMode None
```

The CA needs a reliable way to publish CRLs to the web server. The SMB share gives the CA a controlled publishing path, while read access supports operational needs without

over-permissioning.

Step 12 – Set NTFS permissions on the CRL folder

```
$ACL = Get-Acl "C:\inetpub\cdp\crl"
$AccessRule = New-Object System.Security.AccessControl.FileSystemAccessRule `
    ("Cert Publishers","FullControl","ContainerInherit,ObjectInherit","None","Allow")
$ACL.SetAccessRule($AccessRule)
Set-Acl -Path "C:\inetpub\cdp\crl" -AclObject $ACL
```

Shares alone aren't enough, NTFS must allow the CA (via the AD group *Cert Publishers*) to write and update CRLs. Without this, CRL publishing fails and certificate validation will eventually break.

Whoop, whoop, another step complete! At this point, the PKI Web Server is fully prepared. It can serve CRLs, host policy information, and support the trust chain that will be built next.

One more thing: making sure "trust" actually uses Kerberos

Required permissions

- Domain Admins
- Delegated permission to modify the servicePrincipalName attribute on the web server computer object

Why: SPNs are attributes on computer objects in Active Directory. By default, only Domain Admins can modify them, but this can be safely delegated.

After setting up the PKI web server, everything appears to work as expected. CRLs are reachable, permissions are correct, and the share is accessible via \\trust.domain.suffix. Yet a quick check with `klist`, to enumerate the Kerberos tickets, reveals something subtle, no Kerberos ticket is issued.

When accessing a server via its hostname, Kerberos is configured automatically. But "trust.domain.suffix" is a DNS alias, and Kerberos has no way of knowing that this name belongs to the web server unless it's explicitly told so. No matching SPN means no Kerberos, and Windows falls back to NTLM, bad idea. The solution is straightforward: add CIFS (meaning SMB) SPNs for the alias to the computer account of the web server.

```
setspn -S cifs/trust LAB-WEB-01
setspn -S cifs/trust.domain.suffix LAB-WEB-01
```

Using `-S` ensures duplicate SPNs are detected and avoided.

If you access your CRL share via a DNS alias, make sure Kerberos can actually use it. Without the correct CIFS SPNs, Windows will quietly fall back to NTLM, something you don't want in any infrastructure.

A small detail, but one that makes the difference between *working* and *designed properly*.

In the next part, [Part 3](#), I'll move to the most sensitive component of the entire setup: the [Offline Root Certificate Authority](#).

I'll cover:

- Hardening the server
- Applying BitLocker
- Creating the Root CA
- Preparing it to sign the Enterprise CA

See you there!

References

[The Microsoft Root Certificate Program](#)

[PKI – Part 1: Introduction to Public Key Infrastructure](#)

[PKI – Part 2: Choosing the key length and algorithm](#)

[PKI – Part 3: The role of hash functions in PKI](#)

[PKI – Part 4: Understanding Cryptographic Providers](#)

[PKI – Part 5: Creating Unique Object Identifiers \(OIDs\)](#)

[PKI – Part 6: Demystifying the CAPolicy.inf file](#)